nsparent about the security bugs it has fixed. This transparency creates opportunities for learning, upskilling, and breaking into the InfoSec industry.

- Anyone can learn how to identify and/or fix security issues in a real product.
- AppSec job candidates and new team members get insight into the bugs we face, and our processes.

The reproducible vulnerabilities are structured with expandable hint sections so that you can adapt the content to your desired challenge level. We want it to be approachable to anyone.

If this sort of thing interests you, we encourage you to check for open roles in our Security Team and to participate in our Bug Bounty program on HackerOne (it pays real money!).

Important Information & FAQ

Warning

Installing software with known vulnerabilities carries inherent risk. Do not allow untrusted connections. Ideally run vulnerable instances locally or in a cloud environment allow-listed to your home IP only. Remove your practice instances when you have finished with them.

Won't this page help attackers?

That is not the intent of this page. The intent of this page is to educate both team members and those in the community who are interested in learning about security.

GitLab's disclosure policy is to make patched vulnerabilities public after 30 days. See our process for disclosing security issues. These disclosed issues include steps to reproduce, often include videos or screenshots, and links to the code that patches the issue.

Disclosed vulnerabilities, including those listed here, are already publicly accessible in our issue tracker. This page will never give more assistance or more detail than already exists in those public issues.

What if I find something odd?

If you discover a security issue that still affects current versions of GitLab, for example an incomplete fix, please follow the steps in our Responsible Disclosure Policy.

Reproducible Vulnerabilities

Stored XSS in 15.0

In GitLab 15.0 a malicious user could create a stored XSS payload. See if you can figure out how.

Installation

1. Follow the steps to install a Docker version of GitLab. Depending on your setup, the command will look something like:

shell
sudo docker run --detach \
  --hostname gitlab.example.com \
  --publish 8929:80 --publish 8922:22 \
  --name gitlab15.0.0 \
  --volume $GITLABHOME/config:/etc/gitlab \
  --volume $GITLABHOME/logs:/var/log/gitlab \
  --volume $GITLABHOME/data:/var/opt/gitlab \
  --shm-size 256m \
  gitlab/gitlab-ee:15.0.0-ee.0

After installation is complete you can get the root Administrator user password with docker exec -it gitlab15.0.0 grep 'Password:' /etc/gitlab/initialrootpassword.

1. Register a user to be the attacker
1. Sign in as the Administrator and approve the new user. Sign out.
1. Sign in as the attacker
1. Good hunting!

Bug hunting

Cross-Site Scripting, or XSS, is an attack where attacker-defined javascript is injected into a page the victim is browsing. This can let an attacker impersonate a victim and click on things, submit forms, view data, watch what is typed, and more.

{{% details summary="Hint 1 - Where to start looking" %}}
This XSS vulnerability existed in our CRM feature.

> With customer relations management (CRM) you can create a record of contacts (individuals) and organizations (companies) and relate them to issues.

<https://docs.gitlab.com/ee/user/crm/>
{{% /details %}}

{{% details summary="Hint 2" %}}
One or more of the customer contact fields was susceptible to stored XSS.
{{% /details %}}

{{% details summary="Hint 3" %}}
Instead of giving a contact a real name, put some javascript in there. Can you find anywhere that fails to properly escape the javascript?
{{% /details %}}

{{% details summary="Hint 4" %}}
The contact fields pop up as previews in Issue description and comments when you use the /addcontacts quick action.

{{% /details %}}

{{% details summary="Just tell me how" %}}
Follow the steps to reproduce written by cryptopone on our GitLab Issue:
<https://gitlab.com/gitlab-org/gitlab/-/issues/363293>
{{% /details %}}

{{% details summary="Take it further" %}}
Once you've got a stored XSS payload executing, what can you can do?

Can you elevate your privileges to an administrator, if the victim of your XSS is an administrator?
{{% /details %}}

Vulnerability Details

{{% details summary="Click to expand" %}}
> An issue has been discovered in GitLab affecting all versions starting from 15.0 before 15.0.1. Missing validation of input used in quick actions allowed an attacker to exploit XSS by injecting HTML in contact details. This is a high severity issue (CVSS:3.0/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N, 8.7). It is now mitigated in the latest release and is assigned CVE-2022-1948.
>
> Thanks cryptopone for reporting this vulnerability through our HackerOne bug bounty program.
{{% /details %}}

Remediation

Once you've reproduced the bug, have a go at fixing it locally.

Done that? Now compare your proposed change to our patch(es).

{{% details summary="The fix" %}}
We took multiple steps to holistically address this vulnerability:

- We escaped the first and last name in the following patch: <https://gitlab.com/gitlab-org/gitlab/-/commit/e61e9b9434e2198c4c1d5cf6b4531eb4323c3575>
- We made AppSec required approvers of subsequent changes to the affected files in <https://gitlab.com/gitlab-org/gitlab/-/merge_requests/88419>
- We added SemGrep rules to detect and comment on MRs which might introduce XSS with <https://gitlab.com/gitlab-com/gl-security/product-security/appsec/sast-custom-rules/-/blob/main/appsec-pings/rules.yml#L65-84>
{{% /details %}}

Links

- GitLab Issue: <https://gitlab.com/gitlab-org/gitlab/-/issues/363293>
- Patch: <https://gitlab.com/gitlab-org/gitlab/-/commit/e61e9b9434e2198c4c1d5cf6b4531eb4323c3575>
- Release Post: <https://about.gitlab.com/releases/2022/06/01/critical-security-release-

gitlab-15-0-1-released/>
- CVSS and Bounty: CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N (8.7 High / $13,950.00)
- CVE: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-1948>
- Learn more about XSS:
  - <https://owasp.org/www-community/attacks/xss/>
  - <https://portswigger.net/web-security/cross-site-scripting>

Denial of Service in 14.3.5

On GitLab installations before 14.3.6, a malicious actor could perform a Denial of Service attack by submitting crafted issues, MRs, or comments. See if you can figure out how.

Installation

1. Follow the steps to install a Docker version of GitLab. Depending on your setup, the command will look something like:

shell
sudo docker run --detach \
  --hostname gitlab.example.com \
  --publish 8929:80 --publish 8922:22 \
  --name gitlab14.3.5 \
  --volume $GITLABHOME/config:/etc/gitlab \
  --volume $GITLABHOME/logs:/var/log/gitlab \
  --volume $GITLABHOME/data:/var/opt/gitlab \
  --shm-size 256m \
  gitlab/gitlab-ee:14.3.5-ee.0

After installation is complete you can view the root user password with docker exec -it gitlab14.3.5 grep 'Password:' /etc/gitlab/initialrootpassword. You can use this password to login to the root user on your GitLab instance.

Good hunting!

Bug hunting

Denial of service (DoS) attacks make websites unusable for other users. Typically this happens when a server is so busy dealing with attackers requests that it doesn't have capacity to respond to normal users.

When using GDK locally, open your CPU & Memory resource monitor and look for abnormally high and sustained usage, particularly of the rails-web and rails-background-jobs processes. If you're running GitLab in a docker container, try a tool like ctop.

{{% details summary="Hint 1 - where to start looking" %}}
This DoS involved user content, for example issue descriptions or comments. These can be made via the user interface, or via the API.
{{% /details %}}

{{% details summary="Hint 2" %}}
This researcher found a regex-based DoS by reading the code GitLab used to parse front-matter. <https://gitlab.com/gitlab-org/gitlab/-/blob/6f10f768c9cc2d131c056289f58519cf9cae79fa/lib/gitlab/frontmatter.rb>
{{% /details %}}

{{% details summary="Hint 3" %}}
Search for a flaw called "Catastrophic Backtracking".
{{% /details %}}

{{% details summary="Hint 4" %}}
What happens when you use one of the front matter delimiters followed by content? What happens as that content grows in length? Keep an eye on your process monitor.
{{% /details %}}

{{% details summary="Just tell me how" %}}
Follow the steps to reproduce written by hashkitten on our GitLab Issue: <https://gitlab.com/gitlab-org/gitlab/-/issues/340449>
{{% /details %}}

Vulnerability Details

{{% details summary="Click to expand" %}}
> An issue has been discovered in GitLab CE/EE affecting all versions starting from 12.10 before 14.3.6, all versions starting from 14.4 before 14.4.4, all versions starting from 14.5 before 14.5.2. A regular expression used for handling user input (notes, comments, etc) was susceptible to catastrophic backtracking that could cause a DOS attack. This is a medium severity issue (CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L, 4.3). It is now mitigated in the latest release and is assigned CVE-2021-39933.
>
> Thanks @hashkitten for reporting this vulnerability through our HackerOne bug bounty program.
{{% /details %}}

Remediation

Once you've reproduced the bug, have a go at fixing it locally. Then compare your proposed change to our patch(es).

{{% details summary="The fix" %}}

- We fixed the frontmatter regex: <https://gitlab.com/gitlab-org/gitlab/-/commit/8b30fedf2bea8713bc735638ae63a09f3e4faba1>
- We added timeouts to our rendering pipelines:
  - <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/102819>
  - <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/104779>
{{% /details %}}

Links

- GitLab Issue: <https://gitlab.com/gitlab-org/gitlab/-/issues/340449>

- Release Post: <https://about.gitlab.com/releases/2021/12/06/security-release-gitlab-14-5-2-released/regular-expression-denial-of-service-via-user-comments>
- CVSS and Bounty: CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:L (4.3 Medium / $610.00 / old bounty range)
- CVE: <https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2021-39933>
- Learn more about Denial of Service:
  - <https://owasp.org/www-community/attacks/DenialofService>

<!-- Additional reproducible vulnerabilities go above this line //-->

Contributing

{{% alert color="danger" %}}
Contributions must only include information that is already publicly available.
{{% /alert %}}

Everyone can contribute to this page - that includes you! You can start by clicking "Open in Web IDE" in the sidebar on the right.

First, find an interesting publicly disclosed vulnerability by looking at our public and closed vulnerability issue list or our security release blog posts. Choose a vulnerability that was fixed in any release prior to the latest security release.

Open a Merge Request to this page, mention @gitlab-com/gl-security/product-security/appsec. It should include:

- A title and non-revealing summary of the vulnerability
- Steps to install the vulnerable version
  - Ideally <https://docs.gitlab.com/ee/install/docker/index.htmlinstall-gitlab-using-docker-engine> with a specific version number
  - More complex issues might require a specific installation method, like the Omnibus Linux package.
- A series of progressively revealing hints, so people can try to hunt for the bug themselves but get help if needed. (Remember GitLab is a big product!).
- A link to the GitLab issue, for those who want to follow the original HackerOne report's steps to reproduce.
- A link to the MR(s) that fixed it.

Here's a template you can use:

markdown
Short title goes here

On Free/Premium/Ultimate installations before X.Y.Z, a malicious user could attack type. See if you can figure out how.

Installation

<!-- Steps to install a vulnerable version //-->

Bug hunting

{{% details summary="Hint 1 - where to start looking" %}}
<!-- Something that gets people looking in the right place //-->
{{% /details %}}

{{% details summary="Hint 2" %}}
<!-- Another hint. Add as many hints as you want, using already public data. //-->
{{% /details %}}

{{% details summary="Just tell me how" %}}

Follow the steps to reproduce written by HANDLE on our GitLab Issue:
<https://gitlab.com/gitlab-org/gitlab/-/issues/XXXXXX>

{{% /details %}}

Vulnerability Details

{{% details summary="Click to expand" %}}

<!-- Paste the text from the security release post. Adapt if needed. //-->

{{% /details %}}

Remediation

Once you've reproduced the bug, have a go at fixing it locally.

Then compare your proposed change to our patch(es).

{{% details summary="The fix" %}}

<!-- Link to the patch, with optional description //-->

{{% /details %}}

Links

<!-- Links go here //-->

SECTION: security/product-security/application-security/responding-customers-scan-review-requests.md

We scan our own product using our security scanners. Our Engineering teams are remediating vulnerabilities detected by our scanners on a regular basis. This is done when a patch is available and for vulnerabilities that can be exploited in our context.

We often receive inquiries from customers regarding potential vulnerabilities detected by their own scanning tools or those integrated into GitLab. We value our customers' trust in our expertise to assess these issues. However, given the volume of requests we receive and our limited resources, we kindly request cooperation in performing a reasonable level of initial review before seeking our input. These proactive efforts will enable us to focus on addressing the most critical vulnerabilities efficiently.

Scope

GitLab Images

We are accepting requests to review vulnerabilities detected in the latest version of the following GitLab images:

- gitlab/gitlab-ee:latest (Docker Hub)
- gitlab/gitlab-runner:latest (Docker Hub)

Ensure you are scanning an image running on the latest release or previous two monthly release version. You can consult our maintenance policy for more details on supported versions for security backports. We will typically not be able to review scanner findings for codebases and image versions outside of the maintenance policy.

What scanners results are we accepting?

We are accepting the following scanner results:

- SAST
- DAST
- Dependency scanning
- Container scanning
- Secret detection

Results from our scanners are accepted for review as long as you have triaged the results first.

If you are using third-party security scanners, we ask that you provide the following information for each vulnerability submitted to us:

- An assessment and analysis of the vulnerability.
- Provide a step-by-step guide for the vulnerability to show how it may be exploited in your instance and environment.

Please be sure to add any helpful context to your requests including environment details and specific security risks that you're concerned about. For each risk identified, please provide some explanation and reasoning to help us better understand your concerns so that we can respond in an effective manner.

The format needs to be in an editable file format, for example in CSV, JSON or in an internal spreadsheet.

Will GitLab review scanner results for vulnerabilities of all severity? {review-scanner-results}

No. We will only be able to review scanner results for critical and high vulnerabilities. We will not be able to review scanner results for other severities such as medium, low or informational findings.

Initial self triage

Security scan results are prone to false positives and reported even if vulnerabilities are not exploitable within the application context, making them time consuming to triage. Before submitting your scan results to us, we ask you to perform some initial triage:

- Only submit findings for critical and high vulnerabilities.
- Make sure the vulnerability impacts a component or code within the GitLab directory. GitLab installs required components within that directory. Vulnerabilities for components outside of the GitLab directory will not be investigated as they are not used.
- Vulnerabilities for components that you have installed, either inside or outside of the GitLab directory, will not be investigated.

SLO

Our goal is to reply to customer requests within 10 business days.

Where do I submit scanner findings that adhere to the standards outlined above?

If you are a customer, our Field Security team can open an issue in our issue tracker for you using this template to follow-up on the request internally. Internal team members can directly open issues as long as guidelines outlined above are followed.

SECTION: security/product-security/application-security/stable-counterparts.md

The overall goal of Application Security Stable Counterparts is to help
integrate security themes early in the development process. This is intending
to reduce the number of vulnerabilities released over the long term.
These Stable Counterparts can be found on the
Product Categories page,
next to the "Application Security Engineer" mentions.

Technical Goals

- Assist in threat modeling features to identify areas of risk prior to
  implementation
- Code and AppSec reviews
- Provide guidance on security best practices
- Improve security testing coverage
- Assistance in prioritizing security fixes
- Provide technical guidance on security fixes

Non-technical Goals

- Enable development team to self-identify security risk early
- Help document and solve pain points when it comes to security process
- Identify vulnerability areas to target for training and/or automation
- Assist in cross-team communication of security-related topics
- Assist development teams with security related compliance and regulatory audits

Adding & Updating Stable Counterparts

Stable Counterparts can be added or updated by following these steps:

- (Optional) Organise a 1:1 with the group(s) and App Sec engineer(s) involved to discuss handover, learning opportunities, upcoming priorities, and practicalities like links to GitLab Issue Boards and meeting invitations
- Open an MR to gitlab-com/www-gitlab-com
  - Add or update data/stages.yml, setting the appsecengineer attribute to Firstname Lastname in each group
  - For each App Sec Engineer involved, update the role attribute in data/teammembers/person/DIVISION/LETTER/PERSONNAME.yml to include STAGENAME (GROUPA, GR

Success Stories

There are cases where the Application Security team isn't involved as at the level that the security team would like to, and many factors can lead to that situation. In cases where one is the Stable Counterpart it can be even more difficult to stay up to date on what is happening on each group.

One change in how to approach that for some team members was to setup bi-weekly sync meetings with a member of the engineer team and discuss security related topics. This has started to work on the first meeting and enabled the Application Security to cover important issues that would
not necessarily be seen without this change.

On other cases setting up regular meetings with a dedicated engineer on a very specific topic may also help in staying up to date. In doing that we were able to review issues on new important planned features.

Here are some verbatim answers, capturing how AppSec does (or did) Stable Counterparts in May 2023.

Vitor

- How do/did you get started with your stable counterpart groups?
  - For all the stable counterparts I had, I always started the first discussion with the Product Manager and/or the Engineering Manager. Usually I go for the EMs, but both should provide useful infos about upcoming features and important topics for us to be aware of.
- How do you maintain relationships with them?
  - I had a bi-weekly meeting where we do a sync on the current work, check on questions regarding ongoing reviews, upcoming work. While the first meeting is with the PMs/EMs, the bi-

weekly I had a mix of it being with EMs or Senior/Staff engineers. This is really based on timezone availability where it helps to catchup live. In my opinion this is the most important aspects of our work with them and maintaining a good relationship really helps in being involved as if we were a team member.
- How do you keep track of the work they're doing?
  - I used my 1:1 with my manager to do a status update on the ongoing reviews if it was necessary (usually for important reviews, not for everything), as well as a file where I noted on what I was working (todo list). I could have use a repo on GitLab too with a MD file for example, or used issues to track what I needed to do too.
- How do you get involved in the work they're doing?
  - The bi-weekly syncs are really helpful because they allowed me to get to know the upcoming work and plan accordingly. That allowed me to do few threat models ahead of the review. Another thing is to check for issues using the group's labels combined with Security for example.
- How do you get familiar with the work they've done / their code?
  - It came with time. The more I spend doing reviews for a particular group the more I got comfortable with the code and was familiar with some particular functions. One thing to remember is to not be afraid to ask questions when we do code reviews if the code is not clear to us.
- How do you balance this with other work?
  - My prioritization was the following (by order of importance): Rotations, stable counterpart work, OKRs, other initiatives. If my manager asked me to jump on something top priority then this was obviously taking the priority over anything else (if not on rotation). Of course anything that comes from top management gets prioritized if they are asking us to jump on that with highest priority (examples: an acquisition security review, special ask from my manager and above).

Dominic

- How do/did you get started with your stable counterpart groups?
  - I introduced myself in Slack and had coffee chats with EMs / PMs, though admittedly when the managers change on those teams I didn't do a good job of keeping that up
- How do you maintain relationships with them?
  - I don't do sync meetings at all, I keep an eye in their Slack channels and we talk a lot in issues because they happen to work on a lot of security-sensitive topics. The "maintenance" happens a bit by itself :sweatsmile:
- How do you keep track of the work they're doing?
  - Slack channel for the section, issues filtered by Milestone
- How do you get involved in the work they're doing?
  - Most of the time they ping me, but I keep an eye on issues and MRs I've already been pinged in and keep reading discussions and will often jump in without being re-pinged. I keep up with existing issues and MRs via my emails.
- How do you get familiar with the work they've done / their code?
  - When I'm pinged for a review I take whatever time needed to understand the context. If that means reviewing 5 other MRs that introduced the feature, setting up GDK to play with it and asking a few questions then so be it! I make it clear when I'm not familiar with something and I need ramping up so the expectations are clear.
- How do you balance this with other work?
  - I would say this is my highest priority "non-urgent" work. Incidents and time-sensitive requests will be handled first but other than that I'll do my SC work.

Greg A

- How did you get started with your stable counterpart groups?
  - After taking on a SC, I read the handbook about their direction and what they're working on, and then added my name. From there, I typically set up a chat with the EM to discuss any questions I have, sore spots from a security perspective, and what they hope to gain from the SC program. We articulate expectations to each other and go from there.
- How do you maintain relationships with them?
  - Mostly through Slack. I try to be a constant presence in the smaller teams. For the larger teams, I just lurk in the Slack channel and then communicate one-on-one with the individual who pings me the most for MRs.
- How do you keep track of the work they're doing?
  - Again, mostly through Slack. I'm not incredibly in-tune with every group's work. I do keep tabs on them as most channels have some degree of announcement feature. If I notice that a group has gone completely dark on me, then I'll ping someone to figure out what is going on.
- How do you get involved in the work they're doing?
  - Mostly through pings. The relationship is heavily dependent on them looping me in and me providing a useful answer that will satisfy them for pinging me. Responses to pings are a huge determinant in how the relationship is forming.
- How do you get familiar with the work they've done / their code?
  - I typically don't dive into their work right from the beginning. Most of my familiarization happens during the first couple of MRs and reviews. I take time on my first couple of assignments to explore what they're working on and try to understand the boundaries of their group responsibilities. I ask lots of questions during the first couple of reviews.
- How do you balance this with other work?
  - I consider SC work as my "primary job." When I create a schedule for the day, I prioritize this type of work over other things and really do my best to ensure that I get approvals/comments out as efficiently as possible.

Nikhil

- How do/did you get started with your stable counterpart groups?
  - I introduced myself in the slack channel and their team meeting(async) and scheduled a coffee chat with EM.
- How do you maintain relationships with them?
  - Monthly sync meeting with EM, slack and issue pings. I try to glance through their slack channels from time to time.
- How do you keep track of the work they're doing?
  - Planning issues, issue board and sync meetings.
- How do you get involved in the work they're doing?
  - Most of the time I get pings in issue/MR or slack.
- How do you get familiar with the work they've done / their code?
  - MR reviews and discussion in feature implementation issue.
- How do you balance this with other work?
  - I try to give priority to SC work unless there is an incident or I'm handling a security release.

Nick

- How do/did you get started with your stable counterpart groups?

  - I set the expectation with my groups to please ping me with @ mentions proactively. This leads to MR reviews, epic/issue discussions, and occasionally threat modelling.
  - I ask to be added to team meetings, primarily to get access to the Google Doc.
- How do you maintain relationships with them?
  - At least once a month I'll try and talk to at least one person from my SCs where there is timezone overlap. Usually just a coffee chat that can veer into work if needed.
- How do you keep track of the work they're doing?
  - I join their slack channels, and skim read the conversations each day.
  - I bookmark each team's issue board. Once or twice a month I review the board. I pay particular attention to epics in the design phase (get in early), and to issues which are soon to be / have been merged (better late than never)
  - Each release I'll review their planning issue if they have one
  - On an ad-hoc basis I'll read/watch team meetings, watch demos on YouTube, etc
- How do you get involved in the work they're doing?
  - As above, via reactive pings or proactive board / Slack monitoring
- How do you get familiar with the work they've done / their code?
  - In MR reviews I am comfortable asking questions. I'll try to understand the code, but the team knows it best and asking "silly" questions up front can speed up getting a better understanding
  - After H1 issues, or MR reviews, I might have noticed "code smells". I put time in my calendar to dig into these later. This also helps build familiarity with the code.
  - If I see an opportunity for defense-in-depth, I'll create the MR myself. Again this builds familiarity with the code, and also with the release processes.
- How do you balance this with other work?
  - When on a rotation I largely ignore proactive Stable Counterpart work and solely rely on reactive responses to pings
  - When not on rotation, it's probably 50% SC and the other 50% on everything else (OKRs, handbook updates, working either on the main product or the handbook, G&D, etc)

SECTION: security/product-security/application-security/vulnerability-management.md

Purpose

This procedure applies to vulnerabilities identified in GitLab the product or its dependency projects and ensures implementation of the Vulnerability Management Standard. This procedure is designed to provide insight into our environments, promote healthy patch management among other preventative best-practices, and remediate risk; all with the end goal to better secure our environments and our product.

Scope

The procedure laid out here applies to vulnerabilities identified in GitLab the product or its dependency projects.

Issues in Scope

During the time period defined for the backlog review issues labeled with bug::vulnerability will be reviewed.

Below is a non-exhaustive list of what may be in scope:

- GitLab (gitlab-org/gitlab)
  - HackerOne
  - bug::vulnerability
- Gitaly
- Pages
- Runner
  - HackerOne
  - bug
- Shell
- Customers
- GitLab Dedicated

Roles & Responsibilities

Role	Responsibility
Appsec Engineers	The stable counterpart for each development team will be responsible for ensuring the backlog is accurate and assisting the development team review and triage the backlog for their team. For teams with larger backlogs, help will be requested from other engineers.
Development	Responsible for reviewing the backlog, triaging, and remediating vulnerabilities according to procedure.
Appsec Manager(s)	Responsible for providing support and help make determinations for findings that need additional input.
Security Leadership	Responsible for making final determination on risk acceptance (exceptions).

Procedure

Vulnerability Reports

Vulnerability reports have many sources but always end up as an issue in GitLab.

Some sources include:

- HackerOne reports, which are imported as GitLab issues in the relevant project (see HackerOne Process)
- Users or customers, often submitted via security@gitlab.com or ZenDesk (which are redirected to HackerOne or the GitLab issue tracker)
- Valid scanner findings, which are imported directly from the vulnerability reports page (see this documentation)
- Confidential issues created in GitLab repositories, typically by users, customers, or GitLab team members

Engaging Product Teams

Once the issue is created, a security engineer will set the correct ~group:: and ~devops:: labels and @-mention the engineering and product managers for the team that owns the affected

feature.

The security engineer can try to find a group based on the last people who modified the code by using the git blame feature or by asking in the development Slack channel.

Occasionally the receiving group won't be the correct one, but the security engineer and the group can work together to find the appropriate group to reassign to. A ~devops:: label is good enough if a ~group:: cannot be determined. In this case, the security engineer should assign the issue to the stage's director/Sr. EM.

If an issue is judged to not be a vulnerability but rather a security enhancement, it will be made public, labeled ~"type::feature" and the prioritization will be left to the product team as with any other regular issue. (See: Vulnerability vs. Feature vs. Bug?). For vulnerabilities, the security engineer will set the ~severity:: and ~priority:: labels and those will determine the due date and SLO for the issue as described in the main Product Security page of the handbook.

If the report is a ~severity::1 report, there is a special procedure to follow to engage SIRT and leadership.

Fixing The Vulnerability

Vulnerabilities in the GitLab web application and all other products bundled with our Omnibus packages are fixed following the patch release process. GitLab has two types of security releases: a regular monthly release, and a critical security release whenever a ~severity::1 issue is reported. See the Security Releases section of the main Product Security handbook page for more information.

If you're working on a vulnerability that you feel could be treated as a security enhancement and skip the security release process, feel free to ping @gitlab-com/gl-security/product-security/appsec.

GitLab products that follow an independent release cycle and are not aligned with the monthly GitLab security release are also required to have a process for developing security fixes and disclosing them to the customers. Either the GitLab patch release process or a recommended bare minimum process as below can be followed:

- Do not fix the security issue in public so that the vulnerability and the fix are not disclosed before it is ready to be publicly disclosed. Use a security mirror project under the private Security group for working on the security issue. In case the product is maintained in a private project then the fixes can be done directly in that project.
- Get the security fix reviewed by Appsec team.
- Request the Appsec team create a CVE request for the fixed vulnerability if the product is released to customers for their own installation.
- Notify customers about security fixes.
- Disclose the security issue following the disclosure policy.

Fixing In Public

The Application Security Team may give approval for a security issue to be addressed in public, either wholly or in part. Public issues and MRs must never include information that should remain confidential. Also, backports are not required for security issues fixed in public. Application Security engineers should:

- First, evaluate whether the entire issue can be reclassified and made public.
- Second, consider creating a public linked issue for specific work items.
- Finally, if the issue must remain confidential and creating additional issues is not desired then update the issue with a clear implementation proposal on what can be done in public and what must remain confidential, and add the ~"security-fix-in-public" label.

The security team monitors for unintended information disclosure via public MRs referencing confidential issues and will delete public branches and MRs that do not follow this process.

Notifying GitLab Users

Fixed vulnerabilities are mentioned in the security release blog posts and it's possible to receive notifications either through email or RSS.

Issues that were handled outside of the security release process can be mentioned in the regular release posts if the product manager deems it appropriate.

Security issues fixed in GitLab products may be mentioned in the security release blog post only if they are released along with the monthly GitLab security release. All security issues that need to be mentioned in the security release blog posts are expected to be linked to the ~"upcoming security release" issue. If linking is not possible, please contact the Security release managers to include security issue in the security release blog post.

GitLab products that follow an independent release cycle and are not aligned with the monthly GitLab security release could make use of CHANGELOG.md file or a separate security advisory file in the project repository to publish details about security fixes in their new releases.

Backlog Review

The application security team will do reviews of the vulnerability backlog to ensure that it accurately reflects the vulnerabilities present in the product. The frequency of these reviews will be determined based on compliance requirements or as determined by the team. The process is intended to determine if open issue are still valid, can be closed, or are eligible for risk acceptance.

Labels

Scoped labels will be used to categorize each issue. The labels are as follows:

- ~security-backlog::valid
- ~security-backlog::closed
- ~security-backlog::reclassified
- ~security-backlog::risk-acceptance
- ~security-backlog::needs-input
- ~security-backlog::review-complete

Issue Review Process

For each issue, it's categorization will need to be determined. The following are guidelines the appsec engineer should follow to determine the appropriate category, and the steps that should be taken.

Review Complete

The ~security-backlog::review-complete label should be applied to issues that have been reviewed to help us filter and track.

Valid

An issue is valid if it is a vulnerability that continues to be present in the product.

- Apply the ~security-backlog::valid label

Closed

An issue is closed if it is determined that the issue has previously been fixed,
is a known duplicate of another issue, or is not longer relevant due to evolution of the product.

- Apply the ~security-backlog::closed label
- Leave a comment stating the determined reason and close the issue.
  - If duplicate: Use the /duplicate <issue> slash command to close issue

Reclassified

An issue is reclassified if it can be considered a ~type::feature, ~type::maintenance, or a non-security ~type::bug, based on the evolution of the product, our threat model, or our understanding of the issue. Use the Vulnerability vs. Feature vs. Bug questions below to guide you.

- Apply the ~security-backlog::reclassified label
- Remove severity and priority if assigned by a Security Engineer
- Either:
  - Remove the ~type::bug label and apply the appropriate label.
  - Remove the ~security label.
- Leave a comment stating the determined reason for the reclassification.
- Review the issue for any confidential information and make it public if possible.

Needs Risk Acceptance

The general guidelines for issues eligible for risk acceptance are laid out below. There is no perfect formula, so
the judgement of the engineer will be relied upon.

- Long-lived, with little evidence of an upcoming fix
- Could it be made public with no impact?

  - Could it be labeled ~"Seeking community contributions"?
- Could it be ~security::reclassified as ~"type::maintenance" by the product managers based on a use case? (e.g. Guest user can see a count of X but don't have access to view individual X)
- Is it S3/P3 or lower?

If an issue is eligible for risk acceptance:

- Add comment stating why the issue should be considered for risk acceptance
- Apply the ~security-backlog::risk-acceptance label
- Security will follow Vulnerability Management Standard Risk Acceptance (Exceptions)

Needs Input

An issue needs input if the appsec engineer cannot determine the appropriate category.

- Apply the ~security-backlog::needs-input label

Vulnerability vs. Feature vs. Bug?

When receiving or reclassifying issues, especially S4/P4s, it can be difficult to determine whether it's a vulnerability, an enhancement, or a bug. Here are some questions which might help:

- Is it exploitable and can it affect the confidentiality, integrity and/or availability of GitLab's systems and data? Can you create a CVSS score for it? Vulnerability
- Have similar issues been treated as vulnerabilities, and does that still make sense? Vulnerability
- Is the issue introducing something new which improves security or defense in depth? Enhancement (probably ~type::feature)
- Is the functionality in question acting as designed, but we're refining that to improve security or defense-in-depth? Enhancement (probably ~type::maintenance)
- Is the functionality not acting as designed, but there's no security impact and you can't create a CVSS for it? Non-security Bug
- Are we removing a vulnerable dependency, or piece of code, which GitLab is/was not using in an exploitable way in any of the currently supported versions? (A "theoretical" / unexploitable vulnerability). Enhancement (probably ~type::maintenance)

If it's unclear, err on the side of caution, and treat the issue as a vulnerability. The Engineering Work Type Classification page may also serve as a guide.

When does a security enhancement become escalated to a vulnerability?

Ensuring that appropriate measures are taken to mitigate the associated risks effectively, a security enhancement must be escalated to a vulnerability when any of the following conditions are met:

- When there's evidence demonstrating that it can be combined with another vulnerability, leading to a Critical or High severity security issue or incident
- When there's evidence that the issue is actively exploited in the wild

Updating CVSS after patch

Occasionally after a release has occurred, a discussion in the AppSec team may lead to a better understanding of a bug and its CVSS. Follow the S1/P1 process if an issue is increased in severity to Critical. Otherwise:

- Update the report in HackerOne, if appropriate, and award any additional amount
- Make an MR in the cves-private project to update the CVE record, and mention @gitlab-org/vulnerability-research
- Make an MR to update the security release blog post
- Identify and open an MR for improvements to the triage or remediation processes, if appropriate

Exceptions

Refer to Vulnerability Management Standard Risk Acceptance (Exceptions)

References

- Vulnerability Management Standard
- Infrastructure Vulnerability Management Procedure
- Bug Bounties

SECTION: security/product-security/application-security/appsec-operations/appsec-ops-index.md

<!-- markdownlint-disable MD052 -->
Last updated: May 27, 2025

The Application Security Subdepartment is made up of two teams, the Secure Development and Design Team and the Product Security Incident Response Team (PSIRT). These two teams work together to anticipate and prevent the introduction of vulnerabilities during design and development, as well as identify, assess, and respond to security vulnerabilities discovered in GitLab products and services.

Helpful Quicklinks for GitLab Engineers

- Application Security Reviews
- Application Security Stable Counterparts
- Threat modeling
- Backlog reviews: When necessary a backlog review can be initiated, please see the Vulnerability Management Page for more details.
- GitLab AppSec Inventory
- Responding to customers security scanners review requests
- Root Cause Analysis for Critical Vulnerabilities

Learn how to identify or remediate security issues using real examples with GitLab's Reproducible Vulnerabilities.

Learn how GitLab is implementing Reproducible Builds for our build processes.

Learn more about the automation initiatives that the Application Security team uses on the Application Security Automation and Monitoring page

GitLab Secure Tools coverage

As part of our dogfooding effort, Secure Tools are set up on many different GitLab projects (see our policies. This list is too dynamic to be included in this page, and is now maintained in the GitLab AppSec Inventory.

Projects without the expected configurations can be found in the inventory violations list (internal link).

Useful resources for AppSec engineers

Application Security Engineer Runbooks

Application Security Engineer Job Families

PTO

Team members that are taking PTO for 5 days or more must both discuss time off with their manager prior to scheduling to ensure visibility and adequate team operational coverage and create a PTO coverage issue to organize their coverage during their time off. The PTO coverage issue should :

- List any potential requests that could come to the team while on PTO
- The team member taking PTO should organize their work accordingly and ensure the PTO coverage issue contains the context required to handle the work
- Assign primary and secondary responsible team members

AppSec team members should add any important information related to the work they are covering for the person on PTO and AppSec manager(s) should add any important announcement to see upon their return.

Team Bookmarks

- The AppSec private group that contains other private subgroups and projects
- The appsec-lab group on Staging. This has an Ultimate license.
- Bug bounty council search
- Upcoming patch release
- GitLab Project Security dashboard
- Security issue board that tracks ongoing issues (HackerOne and others)
- The latest releases
- Overview of a project member permissions
- The DevOps stages and their different groups. This page contains information on the development teams, their areas of focus, and their team members as well as the AppSec stable counterparts. It is used to assign issues to the stable counterparts.
- The product features listed by groups that own them

- List of merged security issues in gitlab-org. Note: It can include results from the security mirror gitlab-org/security/.
- Application Security KPIs & Other Metrics - Embedded KPIs which can be filtered by section, stage, or group.
- Milestone Planning - The GitLab Application Security team plans work based around Milestones, see this page for a description of that process

The list above is not exhaustive and is subject to be modified as our processes keep evolving.

Meeting Recordings

The following recordings are available internally only:

- AppSec Sync
- AppSec Leadership Weekly

Content Review and Updates

This page will be reviewed quarterly to ensure alignment with company and divisional priorities, the GitLab Security product roadmap, and relevant business and operational changes. Updates may occur more frequently as business operations evolve.

Next scheduled review: June 30, 2025

SECTION: security/product-security/application-security/appsec-operations/psirt-services.md

<!-- markdownlint-disable MD052 -->
Last updated: May 27, 2025

Product Security Incident Response Team (PSIRT)

The Product Security Incident Response Team (PSIRT) analyzes and validates reports of vulnerabilities in GitLab products and services, and works with GitLab engineers and product teams to remediate and mitigate security vulnerabilities to protect customers. The PSIRT also manages GitLab's Coordinated Vulnerability Disclosure program.

The PSIRT's responsibility includes the fifth and final Secure Developer Experience (SDX) pillar. SDX is a developer UX centered approach to traditional DevSecOps practices.

- SDX: Maintain: establishment of an incident response plan, managing Coordinated Vulnerability Disclosure, bug bounty program administration, and critical product security incident response release and post-release operations.

Helpful Quicklinks

Root Cause Analysis for Critical Vulnerabilities

Application Security Engineer Handling priority::1/severity::1 Issues

Application Security Engineer Working With SIRT

CVSS Calculation

General process for the application security team in patch releases

HackerOne Process

Handling unintended vulnerability disclosures

How to handle upstream security patches

Learn how to identify or remediate security issues using real examples with GitLab's Reproducible Vulnerabilities.

Learn how GitLab is implementing Reproducible Builds for our build processes.

How to Contact the Product Security Incident Response Team

- Mention @gitlab-com/gl-security/product-security/psirt on GitLab
- Ask in sec-appsec and mention @psirt-team on Slack
- For cross team collaboration improvement opportunities, use this template for collaboration improvement opportunities

Content Review and Updates

This page will be reviewed quarterly to ensure alignment with company and divisional priorities, the GitLab Security product roadmap, and relevant business and operational changes. Updates may occur more frequently as business operations evolve.

Next scheduled review: June 30, 2025

SECTION: security/product-security/application-security/appsec-operations/sdd-services.md

<!-- markdownlint-disable MD052 -->
Last updated: May 27, 2025

Secure Design & Development Team Services Overview

The Secure Design & Development Team works with GitLab engineers and product teams to anticipate and prevent the introduction of vulnerabilities during design and development.

Our responsibility includes four of the five Secure Developer Experience (SDX) pillars. SDX is a developer UX centered approach to traditional DevSecOps practices.

- SDX: Learn: security training, governance, policy, documentation, and standards.
- SDX: Design: threat modeling, feature design guidance and consultation, and design reviews.
- SDX: Code: static analysis, software component analysis and supply chain security, use of

approved tools and methodologies in development, deprecation of unsafe functions, etc.
- SDX: Verify: dynamic analysis testing, penetration testing, remediation of critical vulnerabilities, and final security reviews prior to release.

Helpful Quicklinks

- Application Security Reviews
- Application Security Stable Counterparts
- Threat modeling
- Backlog reviews: When necessary a backlog review can be initiated, please see the Vulnerability Management Page for more details.
- GitLab AppSec Inventory
- Responding to customers security scanners review requests
- Root Cause Analysis for Critical Vulnerabilities

Learn how to identify or remediate security issues using real examples with GitLab's Reproducible Vulnerabilities.

Learn how GitLab is implementing Reproducible Builds for our build processes.

Learn more about the automation initiatives that the Application Security team uses on the Application Security Automation and Monitoring page

GitLab Secure Tools coverage

As part of our dogfooding effort, Secure Tools are set up on many different GitLab projects (see our policies). This list is too dynamic to be included in this page, and is now maintained in the GitLab AppSec Inventory.

Projects without the expected configurations can be found in the inventory violations list (internal link).

How to Contact the Secure Design & Development Team

- Find your Stable Counterpart on the Product sections, stages, groups, and categories page
- Mention @gitlab-com/gl-security/product-security/appsec on GitLab
- Submit an issue in the AppSec Team repository
- Ask in sec-appsec or mentioning @appsec-team on Slack
- For cross team collaboration improvement opportunities, use this template for collaboration improvement opportunities

Content Review and Updates

This page will be reviewed quarterly to ensure alignment with company and divisional priorities, the GitLab Security product roadmap, and relevant business and operational changes. Updates may occur more frequently as business operations evolve.

Next scheduled review: June 30, 2025

SECTION: security/product-security/application-security/metrics/_index.md

TBD

SECTION: security/product-security/application-security/metrics/capacity.md

Overview

AppSec manages a wide range of tasks with a high volume of work. This page outlines how we measure the team's capacity to ensure we can effectively handle current workloads and plan for future needs.

Consult our FAQ if you have questions or need to engage with the Application Security team for more specific asks.

What decisions does this data help us make?

Collecting this data helps inform decisions involving the team's capacity and headcount needs. These metrics are only analyzed in aggregate and are not utilized or referenced for evaluating individual team member performance. They are solely used to understand overall team dynamics and requirements.

Where are the charts that are based on this data?

Capacity metrics can be consulted on this Tableau dashboard(Internal)

How often are these metrics reviewed?

At the beginning of each milestone (monthly), it's the responsibility of AppSec team management to lead the review of these metrics with both the AppSec team and ProdSec leadership.

Type of Work Classification

Classifying each type of work helps to distinguish where exactly more capacity or other changes may be necessary.

Table

Label	Description
AppSecWorkType::stable counterpart	Indicates the work was associated to the AppSec stable counterpart duties. MR security reviews are not concerned by this label, use AppSecWorkType::SecurityMRReview instead.
AppSecWorkType::ThreatModel	Indicates the work was associated to the AppSec threat model duties
AppSecWorkType::JihuMRreview	Indicates the work was associated to the AppSec JiHu merge request reviews duties

AppSecWorkType::AppSecReview	Indicates the work was associated to the AppSec reviews duties
AppSecWorkType::KR	Indicates the work was associated to the AppSec OKR/KR duties
AppSecWorkType::SecurityReleaseRotation	Indicates the work was associated to the AppSec Security release task issue duties
AppSecWorkType::VulnFixVerification	Indicates the work was associated to security fix validations during security releases (not the same as security release manager rotation AppSecWorkType::SecurityReleaseRotation label)
AppSecWorkType::HackerOneRotation	Indicates the work was associated to the AppSec HackerOne duties
AppSecWorkType::FieldSecurity	Indicates the work was associated to the request from Field Security (example: customer scan review requests)
AppSecWorkType::VATRotation	Indicates the work was associated to the AppSec Federal AppSec VAT duties
AppSecWorkType::FedAppSecRelCert	Indicates the work was associated to the AppSec Federal AppSec release certification and merge monitor review duties
AppSecWorkType::SecurityMRReview	Indicates the work was associated to the AppSec merge request security reviews (including stable counterpart MR reviews) duties
AppSecWorkType::TriageRotation	Indicates the work was associated to the AppSec Triage Rotation
AppSecWorkType::CustomerEscalation	Indicates the work was associated to a customer escalating a security issue
AppSecWorkType::SIRTandSecurityComms	Indicates the work was associated to a SIRT incidents and/or Security communications work
AppSecWorkType::ToolingsAndMaintenance	Indicates the work was associated to our tools and automation
AppSecWorkType::CrossTeamCollaboration	Indicates the work was associated to cross-team help/collaboration
AppSecWorkType::TeamProjects	Indicates the work was associated to team projects.
AppSecWorkType::CriticalProjects	Indicates the work was associated to critical projects
AppSecWorkType::HackerAdmin	Indicates the work was associated to HackerOne administration
AppSecWorkType::Operational	Should be used for everything else that's not covered by a label above

Work impacted by SIRT incidents

When SIRT incidents happen, this has an impact on our capacity. To evaluate that impact, team members should apply the label ImpactedBySIRTIncidents to the issue.

Dogfooding

Apply Dogfooding label on each feature that we enable through a MR or issue.

Who assigns this label and when?

The AppSec Engineer responsible for the task is expected to assign this label to any Issue or MR as soon as they begin interaction.

Effort Classification

The effort classification is an estimate of the level of effort required to resolve a task, not the amount of actual time spent. The Estimation Guide serves as a reference point and is purely a guideline.

Table

Label	Weight	Classification	Description	Estimation Guide	Example
AppSecWeight::trivial	1	Trivial	Very little effort required	Immediate or near immediate change to resolve the issue	Trivial documentation update
AppSecWeight::small	2	Small	Straight forward change, minimal investigation	~0.5 - 1 days	
AppSecWeight::medium	3	Medium	Some investigation and/or collaboration needed	~1-3 days	
AppSecWeight::large	5	Large	Significant investigation and collaboration needed	~3-5 days	
AppSecWeight::XLarge	8	XLarge	Very complex and requires a major portion of the milestone to resolve	~5-10 days	
AppSecWeight::Needs Refinement	13	Needs Refinement	The issue is overly complex and needs to be promoted to an Epic or broken down into smaller issues	N/A	

Who assigns this label and when?

The AppSec Engineer responsible for the task is expected to assign this label to any Issue or MR after they complete it. In order for an issue to show up in the metrics it needs to have the following labels: ~"AppSecWorkType::<<type>>" ~AppSecWeight::<<weight>> ~"Application Security Team" ~"AppSecWorkflow::complete" as well as a milestone and be closed.

Work flow labels

These labels indicate the current status of the issue.

Table

Label	Description
AppSecWorkflow::planned	Indicates that work has been triaged, scoped, and is ready to be worked on in the assigned milestone.
AppSecWorkflow::in-progress	Indicates the issue is actively being worked on, or the rotation is in progress.
AppSecWorkflow::complete	Indicates the work is done, or the rotation has finished.

Who assigns this label and when?

The AppSec Engineer responsible for the task is expected to assign this label to an issue when work on the issue is started or completed.

Key Performance Indicators

These metrics track our team's capacity to handle critical security workloads.

Merge Request Review Coverage Rate

This KPI tracks our ability to review security-relevant merge requests that introduced a vulnerability, with or without prior security review. It is tracked through a security review miss rate that we target to get as close to 0% as possible, as that would mean that any merge request that was reviewed by the application security team did not end up introducing a vulnerability.

How It's Measured

1. Merge Request Classification Requirements
   - AppSecWorkType::VulnFixVerification must be applied to security fix verification Merge Requests
   - AppSecWorkType::SecurityMRReview must be applied to all other security code reviews, including those performed during triage rotation or as part of the stable counter part MR review.

2. Vulnerability Source Tracking
   - Apply appsec-kpi::vulnerability-introduced label to Merge Requests identified as introducing vulnerabilities

Calculation Method

text
Security Review Miss Rate = (Merged Vulnerability-introducing Merge Requests with Application Security review / Total vulnerability-introducing Merge Requests) 100

Where:

- Total vulnerability-introducing Merge Requests = Merge Requests labeled with appsec-kpi::vulnerability-introduced
- Vulnerability-introducing Merge Requests without Application Security review = appsec-kpi::vulnerability-introduced Merge Requests lacking both AppSecWorkType::SecurityMRReview or AppSecWorkType::VulnFixVerification
- Merged Vulnerability-introducing Merge Requests with Application Security review = appsec-kpi::vulnerability-introduced Merge Requests with either AppSecWorkType::SecurityMRReview or AppSecWorkType::VulnFixVerification

FAQ

What labels should I add to have my work considered in the capacity metrics?

You need to have the Application Security Team, AppSecWorkType::, AppSecWorkFlow:: labels along with the corresponding AppSecWeight:: label and a milestone assigned to the issue.

SECTION: security/product-security/application-security/metrics/results.md

TBD

SECTION: security/product-security/application-security/metrics/risk-patterns.md

TBD

SECTION: security/product-security/application-security/runbooks/_index.md

Note for New team members

Whenever you are on a rotation (HackerOne or Triage Rotation or doing your onboarding process and need help or advice, reach out in the sec-appsec Slack channel or ask during an AppSec Sync meeting. Here are some examples on scenarios where you may need ask or need help:

- You're doing your onboarding tasks, threat modeling, or appsec reviews, and you're stuck on it; or don't know how to tackle something in particular
- You're on ping rotation and you don't know how to deal with a particular situation or what to do with a specific question
- You're on HackerOne rotation and have to deal with a hard report

---
SECTION: security/product-security/application-security/runbooks/bug-hunting-day.md

Bug Hunting Day Process

The Application Security Team has a bug hunting day on the last Friday of every month.

Goal

The objective is to have some time to investigate on something a team member has observed, discussed or noticed during a review but didn't have the time to go into a detailed review or testing.

Bug hunting day is not mandatory. There may be months that are particularly busy and we may not be doing bug hunting days on months like that.

How to get started?

1. Open an issue with the bug-hunting-day template, or click here to create a new one (template already selected).
1. Check the previous issues for some outstanding ideas
1. Put your ideas on the issue or ask the team for topics if you don't have anything in mind
1. Join the zoom meeting time to time during the day to exchange with other team members.

SECTION: security/product-security/application-security/runbooks/cvss-calculation.md

Please refer to the GitLab CVSS Calculator as the single-source-of-truth to determine CVSS scores on GitLab security issues.

SECTION: security/product-security/application-security/runbooks/dependency-review-guidelines.md

This content has been moved to Supply Chain Security for Open Source Dependencies and Libraries.

SECTION: security/product-security/application-security/runbooks/faq.md

This is a curated list of commonly asked questions related to Application Security. If you have a question that is not answered here or in the handbook page please reach out to the AppSec slack channel sec-appsec.

SECTION: security/product-security/application-security/runbooks/federal-appsec-container-scanning-review.md

Certain customers scan containers that GitLab provides for known vulnerabilities and other security concerns. The GitLab Federal Application Security team is responsible for reviewing these results. In many cases, these findings are related to dependencies that need to be updated to address vulnerabilities.

The expectation is that the Federal Application Security Team will verify every finding in each scanned container and make a determination as to whether the finding is a true positive, false positive, or a duplicate. For each one of these determinations, a justification needs to be communicated. These justifications are then reviewed and used to decide whether or not that particular GitLab container is approved for use.

Working the Queue

- Log in to the customer portal. Once logged in, you will see a list of containers that have multiple version tags
- Any container versions that have a finding that need justification (indicated in the Status field) will need to be reviewed and have justifications provided
- For each finding that has a Needs Justification status, determine if it is a true positive, false positive, or duplicate
- Be sure to click the Save Progress button
- Once all justifications have been provided and are ready to be submitted, click the Send for Review button
- Questions or concerns about findings or justifications can be directed to the appropriate

hardening team

True Positives

True positives that are not already being tracked via GitLab issue need to have an issue created in the appropriate repository. The issue description should describe the finding and provide any information relevant to resolving it. This issue needs to then be triaged by applying the appropriate priority/severity labels, a due date, and pinging the responsible product and engineering teams. A justification then needs to be provided that communicates GitLab has confirmed the finding, provides the relevant GitLab issue id, and specifies the expected date of a fix being released via the dropdown.

When trying to determine if something is a true positive:

- Confirm that the scanner finding is actually referencing a dependency that GitLab uses
  - Be mindful that false positives are common because of overlaps in library names
  - The NIST link or other resources can be used to verify what software is impacted by the CVE
- Use the Package Path to determine where the finding is coming from
- Check the appropriate manifest file (Gemfile.lock, go.sum, etc) and to determine if the version mentioned in the scanner finding is actually being used
- Make sure there isn't an existing issue by searching for the CVE identifier and/or the dependency name in the appropriate repository (and perhaps also in gitlab-org/gitlab)
- For some Docker-related findings, consider running bash on the image and checking to see if the impacted library is included by default
  - If the library is included in the base container, an fixed version will likely be included next time the container gets updated
  - In some cases, a version is specified in a GitLab repository and will need to be changed (ex in gitlab-org/build/CNG)

False Positives

If a finding is a false positive, that needs to be communicated in the justification with an explanation as to why it is being considered a false positive.

Duplicates

In some cases, GitLab will already be tracking the finding. These duplicates can be considered as true positives. These will also need a justification to be provided that mentions we are already aware of the finding and should mention the relevant GitLab issue id. Occasionally, the scans will have dupl